ĐẠI HỌC QUỐC GIA HÀ NỘI

ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO THỰC HÀNH 2

Sinh viên thực hiện: Trịnh Quang Sáng

Mã sinh viên: 23020867

Giảng viên hướng dẫn: TS Hoàng Gia Hưng

KS Đỗ Đình Minh

Hà Nội – 2026

Bài 1

Đề bài

Viết module tên `hello`:

- Khi nạp module: in ra log `hello: Hello, Kernel!`.
- Khi gỡ module: in ra log `hello: Goodbye, Kernel!`.

Mã nguồn `hello.c`

```
//Cung cấp các macro module_init, module_exit, MODULE_LICENSE...
#include <linux/module.h>

//Định nghĩa các từ khóa __init và __exit
#include <linux/init.h>

//Cung cấp các hàm tiện ích cơ bản trong kernel
#include <linux/kernel.h>

MODULE_LICENSE("GPL"); //Khai báo giấy phép
MODULE_AUTHOR("TQS"); //Thông tin người viết module
MODULE_DESCRIPTION("Hello LKM"); //Giải thích ngắn gọn chức năng module

//Hàm khởi tạo, được gọi ngay khi module được nạp thành công
static int __init hello_init(void){
    printk(KERN_INFO "Hello, Kernel!\n"); //Ghi log trong kernel với thông tin
    ↪  thông thường
    return 0;
}

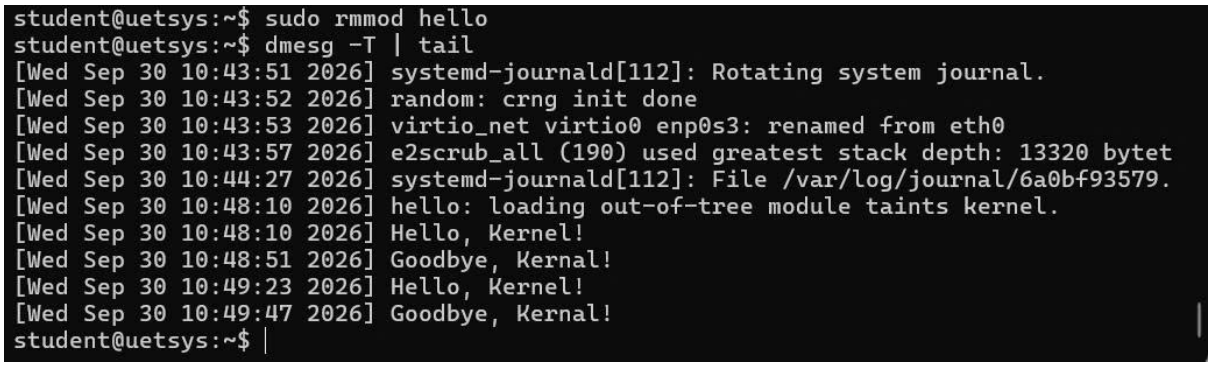
//Hàm thoát, được gọi trước khih module bị gỡ khỏi kernel
static void __exit hello_exit(void){
    printk(KERN_INFO "Goodbye, Kernal!\n");  //Ghi log giá trị thông thường
}

//Đăng ký khai báo entry và exit với kernel
module_init(hello_init);
module_exit(hello_exit);
```

Tệp `Makefile`

```
KDIR ?= $(HOME)/linux-5.15.163
obj-m += hello.o
PWD := $(shell pwd)
all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules
clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
.PHONY: all clean
```

Kết quả thực thi



Kết quả log khi nạp và gỡ module `hello`.

Giải thích

Sau khi module `hello` được nạp bằng lệnh `sudo insmod hello.ko`, hàm `hello_init` sẽ ngay lập tức được gọi và ghi log `Hello, Kernel!`. Và ngay trước khi module bị gỡ bằng lệnh `sudo rmmod hello`, hàm `hello_exit` sẽ được gọi và ghi log `Goodbye, Kernel!`.

Bài 2

Đề bài

Viết module tên `hello_param`:

- Có 2 tham số:
  - `whom` (chuỗi, mặc định `"world"`).
  - `howmany` (số nguyên, mặc định `1`).
- Khi nạp module, in ra `howmany` lần thông điệp:

  `hello_param: Hello, <whom>! [i/N]`
- Khi gỡ module, in thông điệp:

  `hello_param: Bye, <whom>!`

Mã nguồn `hello_param.c`

```
#include <linux/module.h>
#include <linux/init.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("TQS");
MODULE_DESCRIPTION("Hello param LKM");

//Khai báo các tham số truyền vào module
static char *whom = "world";
static int howmany = 1;

//Lập trình module nhận một giá trị đơn lẻ
module_param(whom, charp, 0644); //Biến whom dạng string với toàn quyền truy cập
module_param(howmany, int, 0644);//Biến howmany dạng int với toàn quyền truy cập

//Hàm khởi tạo
static int __init hello_param_init(void){
    int i;
    //Ghi log dữ liệu howmany lần với giá trị của whom và i
    for(i = 1; i <= howmany; i++){
        printk(KERN_INFO "Hello, %s! %d \n", whom, i);
    }
    return 0;
}

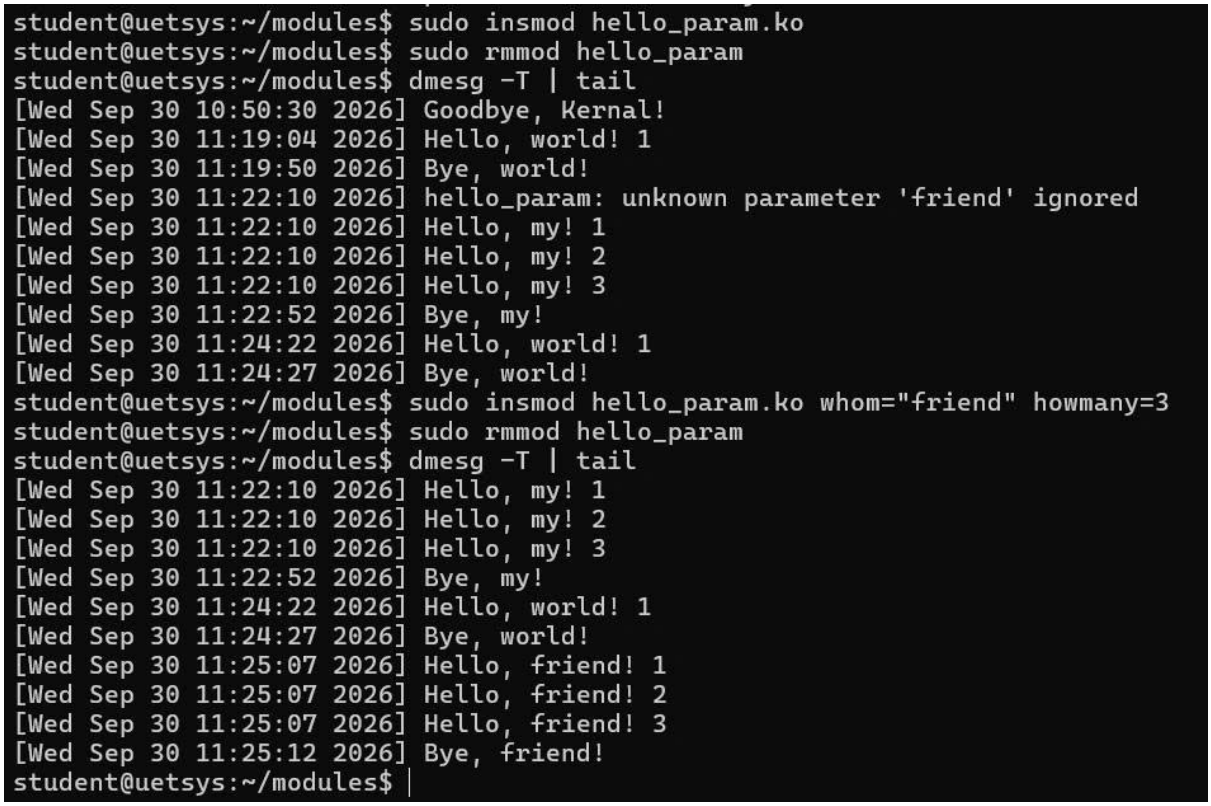
//Hàm thoát
static void __exit hello_param_exit(void){
    //Trước khi gỡ module, ghi log message
    printk(KERN_INFO "Bye, %s!\n", whom);
}

//Đăng ký entry và exit
module_init(hello_param_init);
module_exit(hello_param_exit);
```

Tệp `Makefile`

```
KDIR ?= $(HOME)/linux-5.15.163
obj-m += hello_param.o
PWD := $(shell pwd)
all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules
clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
.PHONY: all clean
```

Kết quả thực thi



Kết quả nạp và gỡ module `hello_param` với tham số mặc định và tham số được truyền vào.

Giải thích

Sau khi nạp module bằng lệnh `sudo insmod hello_param.ko`, hàm `hello_param_init` ngay lập tức được gọi. Tuy nhiên, do không truyền thêm tham số nên chuỗi `whom` và số nguyên `howmany` giữ nguyên giá trị khai báo ban đầu, lần lượt là `"world"` và `1`. Vì vậy, module chỉ ghi log một lần với thông điệp `Hello, world! 1`.

Khi nạp module và truyền tham số bằng lệnh:

```
sudo insmod hello_param.ko whom="friend" howmany=3
```

giá trị của `whom` được đổi thành `"friend"` và `howmany` được đổi thành `3`. Hàm khởi tạo ghi log ba lần theo các giá trị mới, lần lượt là `Hello, friend! 1`, `Hello, friend! 2` và `Hello, friend! 3`.

Bài 3

Ý (a): In các số từ 1 đến n

Đề bài

Viết một module kernel nhận tham số `n` (số nguyên, mặc định bằng `1`). Khi nạp module, in ra các số nguyên từ `1` đến `n`. Khi gỡ module, in ra thông điệp `Done with n=<giá trị>`.

Mã nguồn `print1_n.c`

```
#include <linux/module.h>
#include <linux/init.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("TQS");
MODULE_DESCRIPTION("Print 1 to n LKM");

//Khai báo tham số truyền vào module
static int n = 1;

//Lập trình module nhận giá trị truyền vào
module_param(n, int, 0644);

//Hàm khởi tạo
static int __init print1_n_init(void){
    int i;
    //Ghi log giá trị từ 1 đến n
    for(i = 1; i <= n; i++){
        printk(KERN_INFO "%d\n", i);
    }
    return 0;
}

//Hàm thoát
static void __exit print1_n_exit(void){
    //Trước khi gỡ module, ghi log giá trị cuối cùng
    printk(KERN_INFO "Done with n=%d \n", n);
}

//Đăng ký entry và exit
module_init(print1_n_init);
module_exit(print1_n_exit);
```

Tệp `Makefile`

```
KDIR ?= $(HOME)/linux-5.15.163
obj-m += print1_n.o
PWD := $(shell pwd)
all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules
clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
.PHONY: all clean
```

Kết quả thực thi

```
student@uetsys:~/modules$ sudo insmod print1_n.ko n=5
student@uetsys:~/modules$ sudo rmmod print1_n.ko
student@uetsys:~/modules$ dmesg -T | tail
[Thu Oct  1 07:26:04 2026] 3 and 5
[Thu Oct  1 07:26:04 2026] 4 and 5
[Thu Oct  1 07:26:04 2026] 5 and 5
[Thu Oct  1 07:26:11 2026] Done with n=5
[Thu Oct  1 07:26:44 2026] 1
[Thu Oct  1 07:26:44 2026] 2
[Thu Oct  1 07:26:44 2026] 3
[Thu Oct  1 07:26:44 2026] 4
[Thu Oct  1 07:26:44 2026] 5
[Thu Oct  1 07:26:47 2026] Done with n=5
student@uetsys:~/modules$
```

Kết quả nạp module `print1_n` với tham số `n=5`: module lần lượt in các số từ 1 đến 5 và thông báo `Done with n=5` khi được gỡ.

Giải thích

Sau khi nạp module bằng lệnh `sudo insmod print1_n.ko n=5`, hàm `print1_n_init` được gọi nhờ khai báo `module_init(print1_n_init)`. Giá trị của tham số `n` được đặt thành `5` thông qua `module_param`; sau đó, vòng lặp `for` ghi log 5 lần với các giá trị lần lượt từ `1` đến `n`.

Trước khi module bị gỡ bằng lệnh `sudo rmmod print1_n`, hàm thoát `print1_n_exit` được kích hoạt thông qua `module_exit(print1_n_exit)` và ghi log thông điệp `Done with n=5`.

Ý (b): Tính tổng bình phương từ 1 đến n

Đề bài

Viết một module kernel nhận tham số `n` (số nguyên, mặc định bằng `1`). Khi nạp module, tính tổng bình phương các số từ `1` đến `n` và in kết quả ra log. Khi gỡ module, in ra thông điệp `Done with n=<giá trị>`.

Mã nguồn `print_b.c`

```
#include <linux/module.h>
#include <linux/init.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("TQS");
MODULE_DESCRIPTION("Print 1 to n LKM");

//Khai báo tham số truyền vào module
static int n = 1;

//Lập trình module nhận giá trị truyền vào
module_param(n, int, 0644);

//Hàm khởi tạo
static int __init print_b_init(void){
    int i;
    int result = 0;
    for(i = 1; i <= n; i++){
        result += i * i;
    }
    //Ghi log kết quả tính tổng bình phương từ 1 đến n
    printk(KERN_INFO "Result: %d\n", result);
    return 0;
}

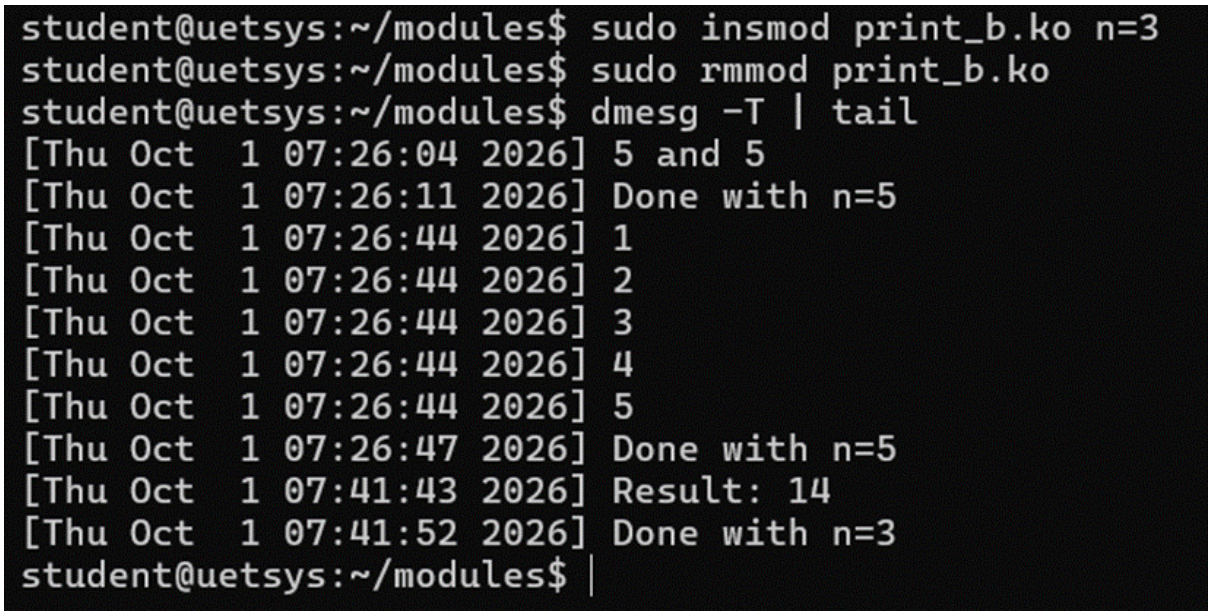
//Hàm thoát
static void __exit print_b_exit(void){
    printk(KERN_INFO "Done with n=%d \n", n);
}

//Đăng ký entry và exit
module_init(print_b_init);
module_exit(print_b_exit);
```

Tệp `Makefile`

```
KDIR ?= $(HOME)/linux-5.15.163
obj-m += print_b.o
PWD := $(shell pwd)
all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules
clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
.PHONY: all clean
```

Kết quả thực thi



Kết quả nạp module `print_b` với tham số `n=3`: tổng bình phương từ 1 đến 3 bằng `14`; khi gỡ module, log hiển thị `Done with n=3`.

Giải thích

Sau khi nạp module bằng lệnh `sudo insmod print_b.ko`, hàm `print_b_init` được gọi nhờ khai báo `module_init(print_b_init)`. Giá trị của tham số `n` được đặt thành `3` thông qua `module_param`, sau đó ghi log kết quả là tổng bình phương từ 1 đến 3 là `Result: 13`.

Trước khi module bị gỡ bằng lệnh `sudo rmmod print_b`, hàm thoát `print_b_exit` được kích hoạt thông qua `module_exit(print_b_exit)` và ghi log với message `Done with n=3`.

Bài 4

Đề bài

Viết module `hello_array`:

- Có tham số `nums` là một mảng số nguyên (tối đa 10 phần tử).
- Khi nạp module, in ra từng giá trị trong `nums` và tổng của chúng.
- Khi gỡ module, in `hello_array: unloaded`.

Mã nguồn `hello_array.c`

```
#include <linux/module.h>
#include <linux/init.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("TQS");
MODULE_DESCRIPTION("Hello Array LKM");

//Khai báo tham số truyền vào module
static int nums[10];
static int num_count = 0;

//Lập trình module nhận giá trị truyền vào
module_param_array(nums, int, &num_count, 0644);

//Hàm khởi tạo
static int __init hello_array_init(void){
    int i;
    int sum = 0;
    //Ghi log từng giá trị trong mảng nums và tính tổng các phần tử
    printk(KERN_INFO "Array elements: ");
    for(i = 0; i < num_count; i++){
        sum += nums[i];
        printk(KERN_INFO "Nums[i] = %d \n", nums[i]);
    }
    //Ghi log tổng các phần tử trong mảng
    printk(KERN_INFO "Sum of array elements: %d\n", sum);
    return 0;
}

//Hàm thoát
static void __exit hello_array_exit(void){
    printk(KERN_INFO "hello_multi: module unloaded\n");
}

//Đăng ký entry và exit
module_init(hello_array_init);
module_exit(hello_array_exit);
```

Tệp `Makefile`

```
KDIR ?= $(HOME)/linux-5.15.163
obj-m += hello_array.o
PWD := $(shell pwd)
all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules
clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
.PHONY: all clean
```

Kết quả thực thi

```
student@uetsys:~/modules$ sudo insmod hello_array.ko nums=1,2,3,5,8
student@uetsys:~/modules$ sudo rmmod hello_array
student@uetsys:~/modules$ dmesg -T | tail
                          Sum of array elements: 19
[Mon Oct  5 04:14:59 2026] hello_multi: module unloaded
[Mon Oct  5 04:17:09 2026] Array elements:
[Mon Oct  5 04:17:09 2026] Nums[i] = 1
[Mon Oct  5 04:17:09 2026] Nums[i] = 2
[Mon Oct  5 04:17:09 2026] Nums[i] = 3
[Mon Oct  5 04:17:09 2026] Nums[i] = 5
[Mon Oct  5 04:17:09 2026] Nums[i] = 8
[Mon Oct  5 04:17:09 2026] Sum of array elements: 19
[Mon Oct  5 04:17:12 2026] hello_multi: module unloaded
student@uetsys:~/modules$
```

Kết quả chạy module `hello_array` với `nums=1,2,3,5,8`; tổng các phần tử bằng 19.

Giải thích

Sau khi nạp module bằng lệnh `sudo insmod hello_array.ko nums=1,2,3,5,8`, hàm `hello_array_init` được gọi nhờ khai báo `module_init(hello_array_init)`, các phần tử trong mảng được truyền vào và số lượng phần tử sẽ được lưu trong biến `num_count`. Khi chạy vòng lặp `for`, sẽ lần lượt in ra giá trị các phần tử và tính tổng các phần tử, và sau vòng lặp sẽ ghi log giá liệu tổng các phần tử trong mảng là `Sum of array element: 19`.

Trước khi module bị gỡ bằng lệnh `sudo rmmod hello_array`, hàm thoát `hello_array_exit` được kích hoạt thông qua `module_exit(hello_array_exit)` và ghi log với message `hello_multi: module unloaded`.

Bài 5

Đề bài

Viết module `hello_multi` gồm hai file nguồn:

- `main.c`: định nghĩa hàm init/exit, gọi hàm từ file phụ.
- `helper.c`: định nghĩa hàm `helper_greet()` in log.
- Khi nạp module, in ra thông điệp `hello_multi: module loaded` và gọi `helper_greet()`.
- Khi gỡ module, in `hello_multi: module unloaded`.

Mã nguồn `helper.c`

```
#include <linux/module.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("TQS");
MODULE_DESCRIPTION("Hello Multi LKM");

//Hàm helper để ghi log message, được gọi từ main.c
void helper_greet(void){
    printk(KERN_INFO "Hello from helper.c!\n");
}
```

Mã nguồn `main.c`

```
#include <linux/module.h>
#include <linux/init.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("TQS");
MODULE_DESCRIPTION("Hello Multi LKM");

//Khai báo hàm helper_greet() được định nghĩa trong file khác
extern void helper_greet(void);

//Hàm khởi tạo
static int __init hello_multi_init(void){
    //In log khi nạp module
    printk(KERN_INFO "hello_multi: module loaded\n");
    //Gọi hàm helper_greet() từ file helper.c
    helper_greet();
    return 0;
}

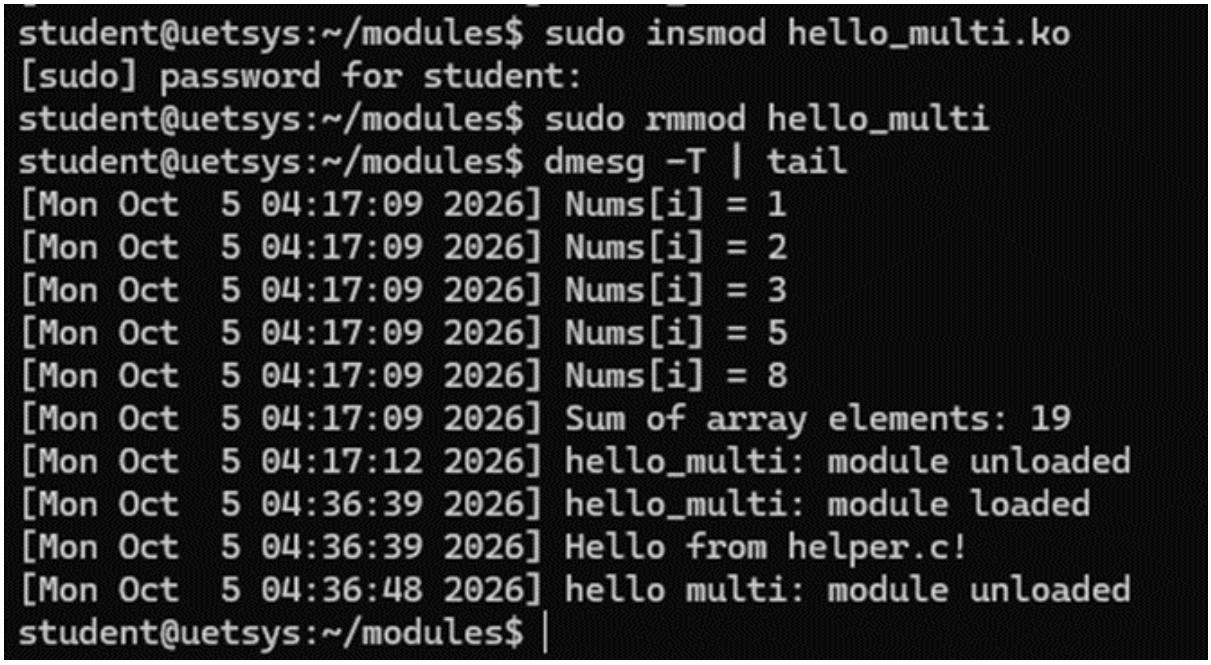
//Hàm thoát
static void __exit hello_multi_exit(void){
    //In log trước khi gỡ module
    printk(KERN_INFO "hello multi: module unloaded\n");
}
```

```
//Đăng ký entry và exit
module_init(hello_multi_init);
module_exit(hello_multi_exit);
```

Tệp `Makefile`

```
KDIR ?= $(HOME)/linux-5.15.163
obj-m += hello_multi.o
hello_multi-objs := main.o helper.o
PWD := $(shell pwd)
all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules
clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
.PHONY: all clean
```

Kết quả thực thi



Kết quả nạp và gỡ module `hello_multi`, kèm thông điệp `Hello from helper.c!`.

Giải thích

Sau khi nạp module bằng lệnh `sudo insmod hello_multi.ko`, hàm `hello_multi_init` được gọi nhờ khai báo `module_init(hello_multi_init)`. Khi đó, hệ thống sẽ ghi log thông báo `hello_multi: module loaded`, đồng thời gọi hàm `helper_greet()` (được khai báo `extern` trong `main.c` và định nghĩa tại `helper.c`) để in tiếp dòng message `Hello from helper.c!`.

Trước khi module bị gỡ bằng lệnh `sudo rmmod hello_multi`, hàm thoát `hello_multi_exit` được kích hoạt thông qua `module_exit(hello_multi_exit)` và ghi log với message `hello multi: module unloaded`.

Bài 6

Đề bài

Viết module `hello_proc`:

- Khi nạp module, in ra:
  - Tên tiến trình (`current->comm`).
  - PID (`current->pid`).
- Khi gỡ module, in `hello_proc: unloaded`.

Ví dụ log:

```
hello_proc: loaded by process "insmod" (pid=1234)
hello_proc: unloaded
```

Mã nguồn `hello_proc.c`

```
#include <linux/module.h>
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/sched.h> //Cung cấp định nghĩa current và struct task_struct

MODULE_LICENSE("GPL");
MODULE_AUTHOR("TQS");
MODULE_DESCRIPTION("Hello Process LKM");

//Hàm khởi tạo
static int __init hello_proc_init(void){
    //Ghi log thông tin về tiến trình hiện tại khi nạp module
    //current là con trỏ trỏ đến struct task_struct của tiến trình hiện tại
    //current->comm: chứa tên của tiến trình đang chạy
    //current->pid: chứa ID của tiến trình đang chạy
    printk(KERN_INFO "hello_proc: loaded by process %s (pid = %d)\n",
    ↪  current->comm, current->pid);
    return 0;
}

//Hàm thoát
static void __exit hello_proc_exit(void){
    printk(KERN_INFO "hello_proc: unloaded \n");
}

//Đăng ký entry và exit
module_init(hello_proc_init);
module_exit(hello_proc_exit);
```

Tệp `Makefile`

```
KDIR ?= $(HOME)/linux-5.15.163
obj-m += hello_proc.o
PWD := $(shell pwd)
all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules
clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
.PHONY: all clean
```

Kết quả thực thi

```
student@uetsys:~/modules$ sudo insmod hello_proc.ko
student@uetsys:~/modules$ sudo rmmod hello_proc
student@uetsys:~/modules$ dmesg -T | tail
[Mon Oct  5 06:33:14 2026] systemd[1]: Started systemd-journald.service - Journal Service.
[Mon Oct  5 06:33:15 2026] systemd-journald[112]: Received client request to flush runtime journal.
[Mon Oct  5 06:33:15 2026] systemd-journald[112]: File /var/log/journal/6a0bf935793c4b52a81d7ebe1a2dd4f1/system.journal.
[Mon Oct  5 06:33:19 2026] virtio_net virtio0 enp0s3: renamed from eth0
[Mon Oct  5 06:33:26 2026] e2scrub_all (189) used greatest stack depth: 13272 bytes left
[Mon Oct  5 06:33:35 2026] systemd-journald[112]: File /var/log/journal/6a0bf935793c4b52a81d7ebe1a2dd4f1/user-1000.jour.
[Mon Oct  5 06:34:25 2026] hello_proc: loading out-of-tree module taints kernel.
[Mon Oct  5 06:34:25 2026] module: x86/modules: Skipping invalid relocation target, existing value is nonzero for type f
[Mon Oct  5 06:36:36 2026] hello_proc: loaded by process insmod (pid = 286)
[Mon Oct  5 06:36:54 2026] hello_proc: unloaded
student@uetsys:~/modules$
```

Kết quả nạp và gỡ module `hello_proc`; tiến trình nạp module là `insmod`, PID bằng `286`.

Giải thích

Sau khi nạp module bằng lệnh `sudo insmod hello_proc.ko`, hàm `hello_proc_init` được gọi nhờ khai báo `module_init(hello_proc_init)`. Hệ thống sẽ sử dụng con trỏ `current` để truy xuất tên tiến trình đang chạy (`current->comm`) và ID của tiến trình đó (`current->pid`), sau đó ghi log với message `hello_proc: loaded by process insmod (pid = 286)`.

Trước khi module bị gỡ bằng lệnh `sudo rmmod hello_proc`, hàm thoát `hello_proc_exit` được kích hoạt thông qua `module_exit(hello_proc_exit)` và ghi log với message `hello_proc: unloaded`.

Bài 7

Đề bài

Viết module `hello_sysfs`:

- Có tham số `val` (số nguyên, mặc định `0`).
- Khai báo `module_param(val, int, 0644)` để cho phép đọc/ghi.
- Khi nạp module, in `hello_sysfs: loaded with val=<giá trị>`.
- Khi gỡ module, in `hello_sysfs: unloaded`.
- Cho phép thay đổi `val` khi đang chạy:

  ```
  echo 42 | sudo tee /sys/module/hello_sysfs/parameters/val
  ```

- Đọc giá trị `val` bằng lệnh:

  ```
  cat /sys/module/hello_sysfs/parameters/val
  ```

Mã nguồn `hello_sysfs.c`

```
#include <linux/module.h>
#include <linux/init.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("TQS");
MODULE_DESCRIPTION("Hello sysfs LKM");

//Khai báo tham số truyền vào module
static int val = 0;

//Lập trình module nhận giá trị truyền vào
module_param(val, int, 0644);

//Hàm khởi tạo
static int __init hello_sysfs_init(void){
    //Ghi log giá trị của tham số val khi nạp module
    printk(KERN_INFO "hello_sysfs: loaded with val = %d \n", val);
    return 0;
}

//Hàm thoát
static void __exit hello_sysfs_exit(void){
    //Ghi log trước khi module bị gỡ bỏ
    printk(KERN_INFO "hello_sysfs: unloaded\n");
}

//Đăng ký entry và exit
module_init(hello_sysfs_init);
module_exit(hello_sysfs_exit);
```

Tệp `Makefile`

```
KDIR ?= $(HOME)/linux-5.15.163
obj-m += hello_sysfs.o
PWD := $(shell pwd)
all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules
clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
.PHONY: all clean
```

Kết quả thực thi

```
student@uetsys:~/modules$ sudo insmod hello_sysfs.ko
student@uetsys:~/modules$ echo 3 | sudo tee /sys/module/hello_sysfs/parameters/val
3
student@uetsys:~/modules$
```

Nạp module `hello_sysfs` và ghi giá trị `3` vào tham số `val` qua sysfs bằng lệnh `echo 3 | sudo tee`.

Giải thích

Sau khi nạp module bằng lệnh `sudo insmod hello_sysfs.ko`, hàm `hello_sysfs_init` được gọi nhờ khai báo `module_init(hello_sysfs_init)`. Hệ thống sẽ tiến hành ghi log với giá trị mặc định ban đầu của biến `val` là `0`.

Tham số `val` được cấp toàn quyền, cho phép người dùng thay đổi giá trị ngay cả khi module đang chạy thông qua lệnh `echo 3 | sudo tee /sys/module/hello_sysfs/parameters/val`. Giá trị của `val` bị ghi đè là `3`.

Cuối cùng, trước khi module bị gỡ, hàm thoát `hello_sysfs_exit` sẽ được kích hoạt thông qua `module_exit(hello_sysfs_exit)` và tiến hành ghi log với message `hello_sysfs: unloaded`.

Bài 8

Đề bài

Viết một module Provider cung cấp hàm `compute_stats` để tính các thông số thống kê cơ bản của một mảng số nguyên gồm:

- Tổng (`sum`).
- Giá trị trung bình (`avg`).
- Giá trị cực đại (`max`).
- Giá trị cực tiểu (`min`).

Kết quả được trả về thông qua một `struct stats_result`. Hàm phải kiểm tra tính hợp lệ của tham số đầu vào và trả lỗi nếu cần.

- Viết một module Consumer sử dụng hàm `compute_stats` đã export từ Provider, gọi thử trên một mảng số nguyên mẫu (ví dụ `{10, 20, 30, 40, 50}`), và in kết quả ra log kernel bằng `printk`.

Mã nguồn `common.h`

```
#ifndef _COMMON_H_
#define _COMMON_H_

//Struct lưu kết quả tính toán
struct stats_result{
    int sum;
    int avg;
    int max;
    int min;
};

int compute_stats(const int *array, int size, struct stats_result *result);

#endif
```

Mã nguồn `provider.c`

```
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include "../common.h"

MODULE_LICENSE("GPL");
MODULE_AUTHOR("TQS");
MODULE_DESCRIPTION("Provider LKM");

//Hàm thực hiện theo yêu cầu đề bài
int compute_stats(const int *array, int size, struct stats_result *result){
    int i;

    if(size <= 0 || array == NULL || result == NULL){
```

```
        return -1;
    }

    result->sum = 0;
    result->max = array[0];
    result->min = array[0];

    for(i = 0; i < size; i++){
        result->sum += array[i];
        if(array[i] > result->max){
            result->max = array[i];
        }
        if(array[i] < result->min){
            result->min = array[i];
        }
    }
    result->avg = result->sum / size;
    return 0;
}

//Đăng ký hàm compute_stats để các module khác có thể sử dụng
EXPORT_SYMBOL(compute_stats);

//Hàm khởi tạo
static int __init provider_init(void){
    printk(KERN_INFO "Provider module loaded\n");
    return 0;
}

//Hàm thoát
static void __exit provider_exit(void){
    printk(KERN_INFO "Provider module unloaded\n");
}

//Đăng ký entry và exit
module_init(provider_init);
module_exit(provider_exit);
```

Tệp `Makefile` của Provider

```
KDIR ?= $(HOME)/linux-5.15.163
obj-m += provider.o
PWD := $(shell pwd)
all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules
clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
.PHONY: all clean
```

Mã nguồn `consumer.c`

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include "../common.h"
```

```
MODULE_LICENSE("GPL");
MODULE_AUTHOR("TQS");
MODULE_DESCRIPTION("Consumer LKM");

//Hàm khởi tạo
static int __init consumer_init(void) {
    int data[] = {10, 20, 30, 40, 50};
    int size = sizeof(data) / sizeof(data[0]);
    struct stats_result result;
    int ret;

    printk(KERN_INFO "Consumer module loaded.\n");

    // Gọi hàm từ provider
    ret = compute_stats(data, size, &result);

    if (ret == 0) {
        printk(KERN_INFO "Sum: %d\n", result.sum);
        printk(KERN_INFO "Avg: %d\n", result.avg);
        printk(KERN_INFO "Max: %d\n", result.max);
        printk(KERN_INFO "Min: %d\n", result.min);
    } else {
        printk(KERN_ERR "Failed to compute stats with error code: %d\n", ret);
    }

    return 0;
}

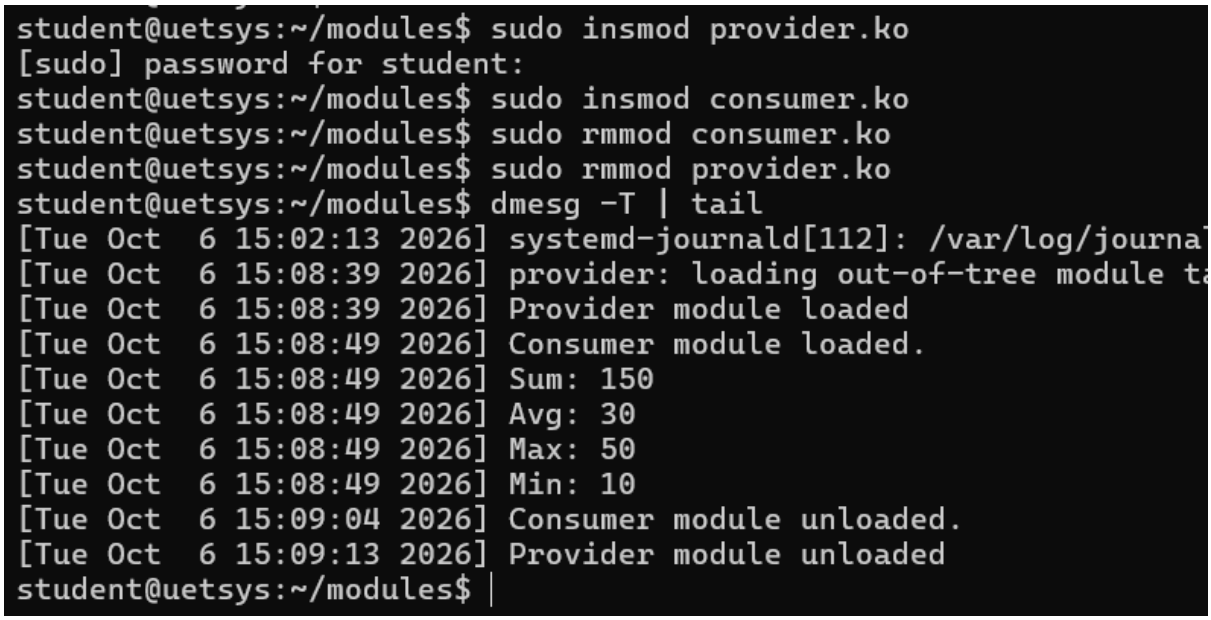
//Hàm thoát
static void __exit consumer_exit(void) {
    printk(KERN_INFO "Consumer module unloaded.\n");
}

//Đăng ký entry và exit
module_init(consumer_init);
module_exit(consumer_exit);
```

Tệp `Makefile` của Consumer

```
KDIR ?= $(HOME)/linux-5.15.163
obj-m += consumer.o
PWD := $(shell pwd)
KBUILD_EXTRA_SYMBOLS := ../Provider/Module.symvers
all:
    $(MAKE) -C $(KDIR) M=$(PWD) modules
clean:
    $(MAKE) -C $(KDIR) M=$(PWD) clean
.PHONY: all clean
```

Kết quả thực thi



Kết quả nạp hai module Provider và Consumer; mảng mẫu cho tổng `150`, trung bình `30`, giá trị lớn nhất `50` và giá trị nhỏ nhất `10`.

Giải thích

Tệp `common.h` được tạo để dùng chung cho hai module, trong đó định nghĩa cấu trúc `struct stats_result` dùng để lưu các kết quả thống kê và khai báo hàm `compute_stats`.

Khi nạp module Provider bằng lệnh `sudo insmod provider.ko`, hàm khởi tạo `provider_init` được kích hoạt và ghi log thông báo module đã được nạp. Module này định nghĩa hàm `compute_stats`, trước tiên kiểm tra tính hợp lệ của mảng đầu vào, kích thước mảng và con trỏ kết quả. Nếu các tham số hợp lệ, hàm duyệt qua mảng để tính tổng, tìm giá trị lớn nhất, giá trị nhỏ nhất và tính trung bình. Macro `EXPORT_SYMBOL(compute_stats)` cho phép các module khác trong kernel nhìn thấy và sử dụng hàm này.

Khi nạp module Consumer bằng lệnh `sudo insmod consumer.ko`, hàm `consumer_init` khởi tạo mảng dữ liệu mẫu `{10, 20, 30, 40, 50}` và gọi hàm `compute_stats` đã được Provider export. Kết quả được ghi chính xác vào log kernel, lần lượt là `Sum: 150`, `Avg: 30`, `Max: 50` và `Min: 10`.